

Data Validation System — Java Swing

Advanced Programming II

THELEDI RAMOUNA MALULEKA

0609091104081

MR OWAMI

GAUTENG CITY COLLEGE

DUE

22 SEPTEMBER 2026

Table of Contents

INTRODUCTION	2
SYSTEM DESIGN	2
FIELDS.....	3
VALIDATION CHECKS.....	4
TESTING	4
Name Testing	4
ID / Student Number Testing.....	6
Age.....	8
Gender Testing	10
Contact Number Testing.....	12
Email Testing.....	14
Complete Valid Form	14
CODE WALKTHROUGH	15
Creating the JFrame	15
Creating Input Fields	16
Gender Selection	16
Name Validation	16
Email Validation	17
Save Button.....	17
Clear Button.....	17
CONCLUSION	18
REFERENCES.....	20

INTRODUCTION

The data validation system is a Java Swing desktop application with the purpose of allowing the system to capture personal information through a graphical user interface and ensure that the information entered by the user is valid before it is accepted. The system captures the name, surname, ID number, contact details, gender, email address and age of users. Each field has appropriate validation checks. These checks help prevent incorrect, incomplete or incorrectly formatted information from being accepted. The application provides immediate feedback when information is invalid.

SYSTEM DESIGN

The system was designed with simplicity and usability in mind. A clear form layout was used so that users can easily understand what information is required. The interface contains a title at the top followed by the input fields. Each field has a descriptive label. Validation feedback is displayed below the relevant field so that users can easily identify and correct errors.

The system contains three main buttons:

- SAVE – validates all information and saves the information when all fields are valid.
- CLEAR – removes all information entered into the form.
- EXIT – closes the application.

The system uses Java Swing components such as JFrame, JPanel, JLabel, JTextField, JRadioButton and JButton.

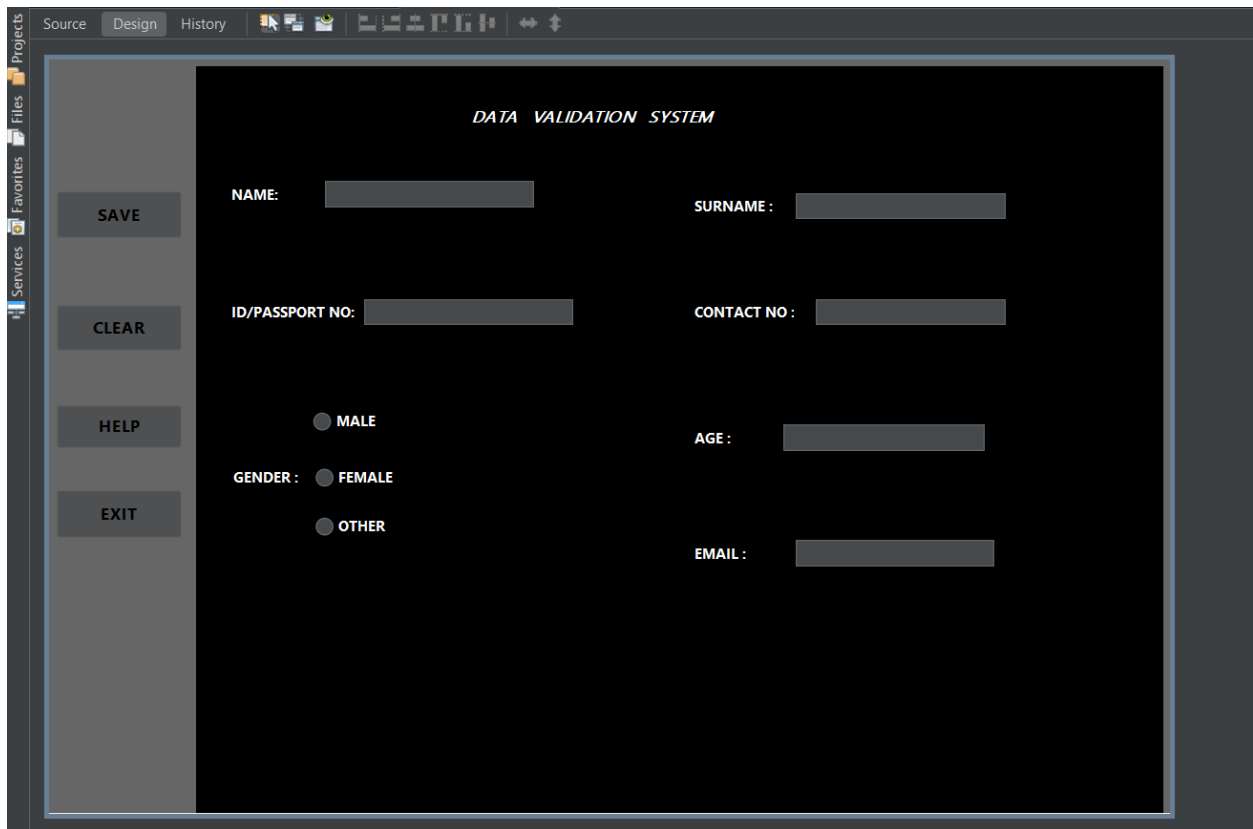


Figure 1: Data validation system interface

FIELDS

- Name and Surname: Captures the user's first name and surname. The field must not be empty and must contain between 3 and 50 characters. Only letters and spaces are accepted.
- ID / Student Number: Captures the user's identification number or student number. It must contain numbers only and must contain between 8 and 13 digits.
- Age: This field captures the user's date of birth. The required format is YYYY-MM-DD. The date cannot be in the future and the calculated age must be between 13 and 120 years.
- Gender: Allows the user to select one of three options: Male, Female or Other. A selection must be made before the information can be accepted.
- Contact Number: Captures the user's contact number. It must contain numbers only and must contain between 10 digits.

- Email Address: Captures the user's email address. It must contain a valid email format.

VALIDATION CHECKS

- Presence check: Makes sure that the user does not leave a required field empty. For example, if the Name and Surname field is empty, the system displays: "THOS FIELD CAN NOT BE EMPTY"
- Type Check: Makes sure that the user enters the correct type of information. For example, the ID and contact number fields require numbers. If a user enters letters into these fields, the system displays an error message.
- Length Check: Ensures that information has an acceptable number of characters. For example, the ID field must contain between 8 and 13 digits. The contact number must contain between 10 and 15 digits.
- Format Check: Ensures that information follows the required pattern. For example, the date of birth must follow the YYYY-MM-DD format and the email address must contain a valid email structure.
- Range Check: Ensures that a numerical value falls within an acceptable range. The system calculates the user's age from the date of birth. The age must be between 13 and 120 years.

TESTING

Name Testing

- **INVALID INPUT**
 - Input: 12345
 - Expected result: The system rejects the input because a name should contain letters and spaces.
 - Error message: "NAME MUST CONTAIL LETTERS ONLY"

The screenshot shows a window titled "DATA VALIDATION SYSTEM". On the left is a sidebar with four buttons: "SAVE", "CLEAR", "HELP", and "EXIT". The main area contains several input fields and validation messages:

- NAME:** A text box containing "12345". Below it, a red message says "MUST CONTAIN LETTERS ONLY".
- SURNAME :** An empty text box. Below it, a red message says "SURNAME MUST BE FILLED IN !!!".
- ID/PASSPORT NO:** An empty text box. Below it, a red message says "ID NUMBER MUST BE FIL...".
- CONTACT NO :** An empty text box. Below it, a red message says "CONTACT CAN NOT BE EMPTY!!".
- GENDER :** Three radio buttons labeled "MALE", "FEMALE", and "OTHER". Below them, a red message says "SELECT A GENDER".
- AGE :** An empty text box. Below it, a red message says "AGE MUST BE FILLED IN !!!".
- EMAIL :** An empty text box. Below it, a red message says "EMAIL CAN NOT BE EMPTY!!".

Figure 2: Invalid name input

- **VALID INPUT**

- Input: John Smith
- Expected result: The system accepts the name.

DATA VALIDATION SYSTEM

NAME: Nxalati

SURNAME:

SURNAME MUST BE FILLED IN !!!

ID/PASSPORT NO:

ID NUMBER MUST BE FIL...

CONTACT NO:

CONTACT CAN NOT BE EMPTY!!

AGE:

AGE MUST BE FILLED IN !!!

EMAIL:

EMAIL CAN NOT BE EMPTY!!

GENDER : ☐ MALE ☐ FEMALE ☐ OTHER

SELECT A GENDER

Figure 3: Valid name input

ID / Student Number Testing

- **INVALID INPUT**
 - Input: ABC123
 - Expected result: The system rejects the input because the ID must contain numbers only.
 - Error message: "ID MUST CONTAIN NUMBERS ONLY."

The screenshot shows a window titled "DATA VALIDATION SYSTEM". On the left is a sidebar with four buttons: "SAVE", "CLEAR", "HELP", and "EXIT". The main area contains several input fields and error messages:

- NAME:** An empty text box with the error message "NAME MUST BE FILLED IN !!!" below it.
- SURNAME:** An empty text box with the error message "SURNAME MUST BE FILLED IN !!!" below it.
- ID/PASSPORT NO:** A text box containing "ABC123" with the error message "MUST CONTAIN NUMBE..." below it.
- CONTACT NO:** An empty text box with the error message "CONTACT CAN NOT BE EMPTY!!" below it.
- GENDER:** Three radio buttons labeled "MALE", "FEMALE", and "OTHER". The "FEMALE" button is selected. Below them is the error message "SELECT A GENDER".
- AGE:** An empty text box with the error message "AGE MUST BE FILLED IN !!!" below it.
- EMAIL:** An empty text box with the error message "EMAIL CAN NOT BE EMPTY!!" below it.

Figure 4: Invalid ID/Passport number

- **VALID INPUT**

- Input: 4433123456789
- Expected result: The system accepts the ID.

DATA VALIDATION SYSTEM

SAVE

CLEAR

HELP

EXIT

NAME:

NAME MUST BE FILLED IN !!!

SURNAME:

SURNAME MUST BE FILLED IN !!!

ID/PASSPORT NO:

CONTACT NO:

CONTACT CAN NOT BE EMPTY!!

AGE:

AGE MUST BE FILLED IN !!!

EMAIL:

EMAIL CAN NOT BE EMPTY!!

☐ MALE

GENDER: ☐ FEMALE

☐ OTHER

SELECT A GENDER

Figure 5: Valid ID/Passport number

Age

- **INVALID INPUT**

- Input: 15
- Expected result: The system rejects the date because it is in the future.
- Error message: "AGE MUST BE BETWEEN 15 AND 120"

DATA VALIDATION SYSTEM

SAVE

CLEAR

HELP

EXIT

NAME:

NAME MUST BE FILLED IN !!!

SURNAME :

SURNAME MUST BE FILLED IN !!!

ID/PASSPORT NO:

ID NUMBER MUST BE FIL...

CONTACT NO :

CONTACT CAN NOT BE EMPTY!!

AGE :

Age must be between 18 and 100

EMAIL :

EMAIL CAN NOT BE EMPTY!!

☐ MALE

GENDER : ☐ FEMALE

☐ OTHER

SELECT A GENDER

Figure 6: Invalid age

- **VALID INPUT**
 - Input: 25
 - Expected result: The system accepts the date if the calculated age is within the allowed range.

DATA VALIDATION SYSTEM

NAME: SURNAME:
NAME MUST BE FILLED IN !!! SURNAME MUST BE FILLED IN !!!

ID/PASSPORT NO: CONTACT NO:
ID NUMBER MUST BE FIL... CONTACT CAN NOT BE EMPTY!!

AGE:

GENDER: ☐ MALE ☐ FEMALE ☐ OTHER
SELECT A GENDER

EMAIL:
EMAIL CAN NOT BE EMPTY!!

SAVE CLEAR HELP EXIT

Figure 7: Valid age

Gender Testing

- **INVALID INPUT**
 - No gender selected.
 - Expected result: The system rejects the form.
 - Error message: "SELECT A GENDER."

DATA VALIDATION SYSTEM

NAME:

NAME MUST BE FILLED IN !!!

SURNAME:

SURNAME MUST BE FILLED IN !!!

ID/PASSPORT NO:

ID NUMBER MUST BE FIL...

CONTACT NO:

CONTACT CAN NOT BE EMPTY!!

AGE:

AGE MUST BE FILLED IN !!!

EMAIL:

EMAIL CAN NOT BE EMPTY!!

GENDER:

☐ MALE

☐ FEMALE

☐ OTHER

SELECT A GENDER

SAVE

CLEAR

HELP

EXIT

Figure 8: No gender selected

- **VALID INPUT**
 - Select: Female
 - Expected results: The gender is accepted

DATA VALIDATION SYSTEM

NAME:
NAME MUST BE FILLED IN !!!

SURNAME :
SURNAME MUST BE FILLED IN !!!

ID/PASSPORT NO:
ID NUMBER MUST BE FIL...

CONTACT NO :
CONTACT CAN NOT BE EMPTY!!

AGE :
AGE MUST BE FILLED IN !!!

EMAIL :
EMAIL CAN NOT BE EMPTY!!

GENDER :

☐ MALE

☒ FEMALE

☐ OTHER

SAVE

CLEAR

HELP

EXIT

Figure 9: Gender selected

Contact Number Testing

- **INVALID INPUT**
 - Input: ABC123
 - Expected Result: The system rejects the number
 - Error Message: "CONTACT NUMBER MUST CONTAIN LETTERS ONLY"

DATA VALIDATION SYSTEM

SAVE

CLEAR

HELP

EXIT

NAME:
NAME MUST BE FILLED IN !!!

SURNAME :
SURNAME MUST BE FILLED IN !!!

ID/PASSPORT NO:
ID NUMBER MUST BE FIL...

CONTACT NO :
CONTACT NUMBER MUST BE BETWEEN 10 AND 15 DIGITS

AGE :
AGE MUST BE FILLED IN !!!

EMAIL :
EMAIL CAN NOT BE EMPTY!!

☒ **MALE**

GENDER : ☐ **FEMALE**

☐ **OTHER**
SELECT A GENDER

Figure 10: Invalid contact details

- **VALID INPUT**
 - Input: 0821234567
 - Expected result: The contact number is accepted

The screenshot shows a window titled "DATA VALIDATION SYSTEM". On the left is a sidebar with four buttons: "SAVE", "CLEAR", "HELP", and "EXIT". The main area contains a form with the following fields and messages:

- NAME:** [Empty text box] with message "NAME MUST BE FILLED IN !!!" below it.
- SURNAME :** [Empty text box] with message "SURNAME MUST BE FILLED IN !!!" below it.
- ID/PASSPORT NO:** [Empty text box] with message "ID NUMBER MUST BE FIL..." below it.
- CONTACT NO :** [Text box containing "0821234567"]
- GENDER :** Three radio buttons labeled "MALE", "FEMALE", and "OTHER". Below them is the message "SELECT A GENDER".
- AGE :** [Empty text box] with message "AGE MUST BE FILLED IN !!!" below it.
- EMAIL :** [Empty text box] with message "EMAIL CAN NOT BE EMPTY!!" below it.

Figure 11: Valid contact details

Email Testing

- **INVALID INPUT**

- Input: johnemail.com
- Expected result: The system rejects the email because it does not contain a valid email format.
- Error message: "ENTER A VALID EMAIL ADDRESS"

- **VALID INPUT**

- Input: john@gmail.com
- Expected result: The email address is accepted.

Complete Valid Form

- When all fields contain valid information, the system accepts the information and displays a successful validation message.
 - Example;
 - Name: John Smith

- ID: 123456789
 - Age: 25
 - Gender: Male
 - Contact: 0821234567
 - Email: john@gmail.com
- The system displays: "All information is valid and has been saved successfully!"

The screenshot shows a Java Swing window titled "DATA VALIDATION SYSTEM". On the left is a vertical sidebar with four buttons: "SAVE", "CLEAR", "HELP", and "EXIT". The main area contains a form with the following fields:

- NAME:** Text input field containing "learato".
- SURNAME:** Text input field containing "maluleka".
- ID/PASSPORT NO:** Text input field containing "1234567890987".
- CONTACT NO:** Text input field containing "1234567899".
- GENDER:** Radio button group with three options: "MALE", "FEMALE" (selected), and "OTHER".
- AGE:** Text input field containing "25".
- EMAIL:** Text input field containing "learatom@gmail.com".

Figure 12: Completed with all valid information

CODE WALKTHROUGH

Creating the JFrame

- The JFrame creates the main window of the application. public class DataValidationSystem extends JFrame { This allows the application to use the Java Swing graphical user interface.

Creating Input Fields

- JTextField components are used to allow the user to enter information.

```
nameField = new JTextField();
```

```
idField = new JTextField();
```

```
dobField = new JTextField();
```

```
contactField = new JTextField();
```

```
emailField = new JTextField();
```

Each text field is connected to a specific piece of personal information.

Gender Selection

- JRadioButton components are used for gender selection.

```
maleButton = new JRadioButton("Male");
```

```
femaleButton = new JRadioButton("Female");
```

```
otherButton = new JRadioButton("Other");
```

A ButtonGroup makes the choices work as one selection group.

Name Validation

- The name validation method checks whether the field is empty, checks its length and checks whether it contains only letters and spaces.

```
if (name.isEmpty()) {  
    nameError.setText("Name is required.");  
    return false;  
}
```

The method returns true when the information passes the validation.

Email Validation

- The email validation uses a pattern to check whether the email has an appropriate structure.

```
String emailPattern =  
    "^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$";
```

If the email does not match the pattern, the user receives an error message.

Save Button

- The Save button validates all fields.

```
boolean nameValid = validateName();  
boolean idValid = validateID();  
boolean dobValid = validateDOB();  
boolean genderValid = validateGender();  
boolean contactValid = validateContact();  
boolean emailValid = validateEmail();
```

The information is only accepted when all validation methods return true.

Clear Button

- The Clear button removes the information entered into the form.

```
private void clearForm() {  
    nameField.setText("");  
    idField.setText("");  
    dobField.setText("");  
    contactField.setText("");  
    emailField.setText("");  
}
```

This allows the user to start again without closing the application.

CONCLUSION

The Personal Data Validation System was successfully designed to demonstrate how Java Swing can be used to develop a desktop application that captures, validates and manages personal information. The main purpose of the project was to ensure that information entered by a user is complete, correctly formatted and within the required limits before it is accepted by the system. The application includes important personal information fields such as name and surname, ID or student number, date of birth, gender, contact number and email address.

During the development of the system, different validation techniques were implemented. These included presence checks, type checks, length checks, format checks and range checks. Each validation method was designed according to the type of information being entered. For example, the name field only accepts letters and spaces, while the ID and contact number fields accept numerical values. The email field checks whether the information follows an appropriate email format, while the date of birth field checks that the date is valid, is not in the future and results in an acceptable age.

One of the challenges experienced during the development of the system was making sure that all fields were validated correctly without causing the program to crash. This required careful use of conditional statements, regular expressions and exception handling. The date of birth validation was particularly important because the system needs to recognise an incorrect date format and also check whether the date is reasonable. Providing clear error messages also helped make the system easier for users to understand and use.

The project also improved my understanding of Java Swing and how different graphical user interface components work together. I gained experience using components such as JFrame, JPanel, JLabel, JTextField, JRadioButton and JButton. I also learned how event listeners can be used to make buttons perform specific actions, such as validating information, clearing the form and closing the application.

Another important part of the project was testing. Testing the system with both invalid and valid information helped identify whether each validation rule was working correctly. Invalid information was used to check whether the system displayed appropriate error messages, while valid information was used to confirm that the system could accept correct data. This demonstrated the importance of testing a program before it is considered complete.

The project also helped develop my problem-solving and programming skills. Instead of simply allowing users to enter any information, the system applies rules to protect the

quality and accuracy of the data. This shows how validation is an important part of software development because incorrect information can affect the reliability of a system.

In conclusion, the Data Validation System provided practical experience in developing a Java Swing application from design through to implementation and testing. It improved my understanding of graphical user interfaces, input validation, error handling, event-driven programming and structured code. The project also demonstrated the importance of creating software that is not only functional but also easy for users to understand. The knowledge gained from this project can be applied to future Java applications and other systems where accurate and reliable user information needs to be captured.

REFERENCES

Oracle (2026) *Java SE API documentation: Swing package*. Available at: <https://docs.oracle.com/en/java/javase/26/docs/api/java.desktop/module-summary.html> (Accessed: 22 September 2026).

Oracle (2026) *Java SE API documentation: JFrame class*. Available at: <https://docs.oracle.com/en/java/javase/26/docs/api/java.desktop/javax/swing/JFrame.html> (Accessed: 22 September 2026).

Oracle (2026) *Java SE API documentation: JTextField class*. Available at: <https://docs.oracle.com/en/java/javase/26/docs/api/java.desktop/javax/swing/JTextField.html> (Accessed: 22 September 2026).

Oracle (2026) *Java SE API documentation: JButton class*. Available at: <https://docs.oracle.com/en/java/javase/26/docs/api/java.desktop/javax/swing/JButton.html> (Accessed: 22 September 2026).

Oracle (2026) *Java SE API documentation: JRadioButton class*. Available at: <https://docs.oracle.com/en/java/javase/26/docs/api/java.desktop/javax/swing/JRadioButton.html> (Accessed: 22 September 2026).

Oracle (2026) *Java SE API documentation: LocalDate class*. Available at: <https://docs.oracle.com/en/java/javase/26/docs/api/java.base/java/time/LocalDate.html> (Accessed: 22 September 2026).

Oracle (2026) *Java SE API documentation: DateTimeFormatter class*. Available at: <https://docs.oracle.com/en/java/javase/26/docs/api/java.base/java/time/format/DateTimeFormatter.html> (Accessed: 22 September 2026).

Oracle (2026) *Java Tutorials: Regular Expressions*. Available at: <https://docs.oracle.com/javase/tutorial/essential/regex/> (Accessed: 22 September 2026).

Mukul Saini Skills (2018) *Java Swing Tutorial | JFrame in Java Swing*. YouTube. Available at: <https://www.youtube.com/watch?v=OtwMij223OY> (Accessed: 22 September 2026).

Mukul Saini Skills (2018) *Java Swing Tutorial | JRadioButton in Java Swing*. YouTube. Available at: <https://www.youtube.com/watch?v=ekW0V3jVMH4> (Accessed: 22 September 2026).

Bro Code (n.d.) *Java GUI Tutorial for Beginners — Swing Graphics*. YouTube. Available at: <https://www.youtube.com/> (Accessed: 22 September 2026).

CodeBeauty (n.d.) *REGEX — How to Match and Search Any Pattern or Validate Input*. YouTube. Available through YouTube. (Accessed: 22 September 2026).

The Net Ninja (n.d.) *Regular Expressions (RegEx) Tutorial*. YouTube. Available through YouTube. (Accessed: 22 September 2026).

GitHub (2026) *GitHub Documentation: About repositories*. Available at: <https://docs.github.com/en/repositories/creating-and-managing-repositories/about-repositories> (Accessed: 22 September 2026).

GitHub (2026) *GitHub Documentation: About README files*. Available at: <https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-readmes> (Accessed: 22 September 2026).

GitHub (2026) *GitHub Documentation: About commits*. Available at: <https://docs.github.com/en/pull-requests/committing-changes-to-your-project/about-commits> (Accessed: 22 September 2026).